

YZM2022 — Data Mining

Homework 2: Text Mining and Classification

Human vs. Machine Generated Text Detection

Tuğana Öykü Yıldız

Student ID: 24018020

Abstract

In this homework I built machine learning models that decide if a text is written by a human or generated by an AI system. I used a dataset of 10,000 texts (5,000 human and 5,000 machine). The text was cleaned with standard NLP steps and converted to numbers using TF-IDF with unigram and bigram features. I tested four feature sizes (500, 2,000, 5,000, 10,000), four classifiers (KNN, Naive Bayes, Decision Tree, Random Forest), and five feature selection options (ReliefF, Chi-Square, Information Gain, ANOVA F-test, plus a no-FS baseline). All experiments used 5-fold stratified cross validation. The best configuration was Random Forest with K=5,000 features and Information Gain selection, reaching $F1=0.8635$ and $AUC=0.9509$.

1. Objective

Recent advances in artificial intelligence have made it possible for systems to generate text that looks very similar to human writing. This creates a real need for tools that can tell human and machine text apart, for example in academic integrity, plagiarism checks, and news verification.

The main goal of this homework is to develop machine learning models that classify whether a given text is generated by an AI system or written by a human. This is a binary classification problem with two labels: human and machine.

2. Dataset

The dataset contains 10,000 English texts in a CSV file with two columns: text (the content) and label_cat (the class label).

- **Total samples:** 10,000
- **Human texts:** 5,000
- **Machine texts:** 5,000
- **Class balance:** Perfectly balanced (50/50), so no oversampling is needed.
- **Missing values:** None

2.1 Exploratory Data Analysis

Before building models, I checked the basic statistics of the texts. The histograms below compare word and character lengths between human and machine texts.

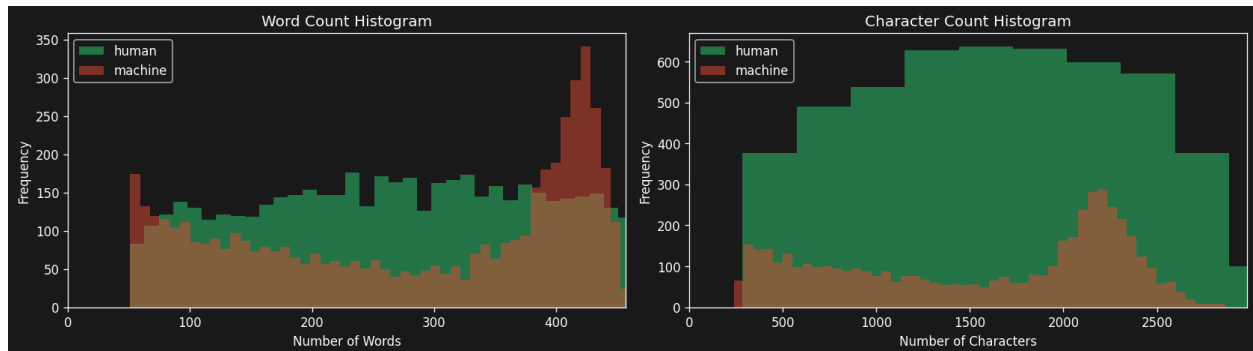


Figure 1. Word count and character count histograms by class.

Two important things can be seen in these histograms:

1. Machine texts have a sharp peak around 400–450 words, which is probably because of the default token limit of the LLMs that produced the data.
2. Human texts have a wider distribution and reach much higher character counts. They are generally longer and more variable in length.

This means that text length itself is already a useful signal that any classifier can pick up on indirectly.

3. Methodology

My pipeline has four main stages: text preprocessing, feature extraction, feature selection, and classification.

3.1 Text Preprocessing

I applied four standard NLP cleaning steps in this order:

1. Lowercasing — convert all text to lower case.
2. Cleaning — remove all non-letter characters (numbers and punctuation) using a regular expression.
3. Tokenization — split each text into individual words using NLTK's `word_tokenize`.
4. Stopword removal & lemmatization — remove common words (the, a, is, etc.) and reduce remaining words to their base form (used → use) with NLTK's `WordNetLemmatizer`. I chose lemmatization over stemming because lemmatization keeps real words while stemming can break them (people → peopl).

After preprocessing, I removed any text that became too short (less than 3 tokens). Only 9 machine samples were dropped, so the final dataset has 9,991 examples.

3.2 Feature Extraction (TF-IDF)

I converted the cleaned texts into numerical vectors using TF-IDF with unigrams and bigrams together. The TF-IDF parameters were:

- **ngram_range=(1, 2):** Both single words and two-word combinations are used as features.
- **max_features=K:** Keep only the top K features by frequency. I tested K = 500, 2,000, 5,000, and 10,000.
- **min_df=2:** Ignore terms that appear in only one document (noise).
- **max_df=0.95:** Ignore terms that appear in more than 95% of documents (too common).
- **sublinear_tf=True:** Use logarithmic frequency scaling to reduce the effect of outliers.

Important note on data leakage: I do not fit TF-IDF on the whole corpus. Instead, the TF-IDF vectorizer is fit on the training fold only inside each cross-validation iteration, and the test fold is just transformed. This avoids leaking test fold statistics (IDF values, max_features selection) into training. The same rule applies to feature selection.

3.3 Classification Algorithms

I used four classifiers, all from scikit-learn:

Algorithm	Parameters	Reason for Selection
KNN	k=5, metric=cosine, n_jobs=-1	Cosine similarity works well for high-dimensional sparse text vectors.
Naive Bayes	MultinomialNB (default)	Classic and fast baseline for word-frequency features like TF-IDF.
Decision Tree	max_depth=20, random_state=42	Limited depth to reduce overfitting on a single tree.
Random Forest	n_estimators=100, max_depth=30, n_jobs=-1, random_state=42	Ensemble of trees, robust to high dimensions and usually the strongest model.

3.4 Feature Selection

ReliefF was the required method. I added three more filter-based methods to compare results, plus a no-FS baseline:

Method	Type	Notes
ReliefF	Instance-based (required)	Looks at nearest same-class and different-class neighbors. Uses skrebate, with subsampling for speed.
Chi-Square	Filter / statistical	Works very well with non-negative TF-IDF features.
Information Gain	Filter / entropy-based	Mutual information between feature and class label.
ANOVA F-test	Filter / statistical	Variance test between classes.
AllFeatures	Baseline (no FS)	TF-IDF max_features only; used to measure FS impact.

For each feature size K, the selector keeps half of the features ($k_{\text{select}} = K/2$). For example, when $K=2,000$, feature selection reduces it to 1,000. Each method is fit only on the training fold.

Note: ReliefF has complexity $O(m \cdot n \cdot d)$. At $d=10,000$ it becomes very slow (around 25–35 minutes per fold). To keep total runtime feasible, I use subsampling (1,500 samples per fit) inside the ReliefF wrapper, and I skip ReliefF entirely when $K=10,000$. This is documented as a limitation, and the other three methods cover the $K=10,000$ case.

4. Evaluation

4.1 Cross Validation

I used 5-fold Stratified Cross Validation. The data is split into 5 equal parts; each part is used once as the test set while the other four are used for training. "Stratified" means each fold keeps the same class balance as the full dataset. The final score for each metric is the mean across the 5 folds.

4.2 Performance Metrics

For each model I report five metrics:

- **Accuracy:** Percentage of correctly classified texts.
- **Precision (macro):** How many of the predicted-positive texts were actually positive.
- **Recall (macro):** How many of the real positive texts were correctly predicted.
- **F1-score (macro):** Harmonic mean of precision and recall.
- **AUC:** Area under the ROC curve, summarizes classifier ranking quality across thresholds.

4.3 Experimental Setup

In total I ran 76 model configurations: 4 feature sizes $\times$ 5 feature selection methods $\times$ 4 classifiers, minus the 4 ReliefF + K=10,000 combinations that were skipped. With 5-fold CV this means 380 individual model trainings. Random seed was fixed to 42 for reproducibility.

5. Results

5.1 Top 10 Configurations

The table below shows the 10 best configurations sorted by F1 score. Every one of them is a Random Forest, which already gives a strong message about which classifier works best.

K	FS Method	Classifier	Acc	Prec	Recall	F1	AUC
5000	InfoGain	RandomForest	0.8644	0.8735	0.8644	0.8635	0.9509
2000	AllFeatures	RandomForest	0.8636	0.8716	0.8636	0.8628	0.9504
2000	InfoGain	RandomForest	0.8601	0.8664	0.8601	0.8595	0.9431
10000	InfoGain	RandomForest	0.8591	0.8691	0.8591	0.8581	0.9487
10000	ANOVA-F	RandomForest	0.8589	0.8690	0.8589	0.8579	0.9457
10000	AllFeatures	RandomForest	0.8580	0.8674	0.8580	0.8571	0.9455
5000	AllFeatures	RandomForest	0.8576	0.8678	0.8576	0.8566	0.9478
2000	Chi2	RandomForest	0.8575	0.8662	0.8575	0.8566	0.9436
2000	ANOVA-F	RandomForest	0.8561	0.8637	0.8561	0.8553	0.9444
5000	Chi2	RandomForest	0.8554	0.8646	0.8554	0.8545	0.9447

Table 1. Top 10 configurations ranked by F1-score. Highlighted row = best overall.

5.2 Pivot Table — F1 Scores Across All Configurations

This pivot table shows F1 score for every (FeatureSize × FS Method × Classifier) combination. The bold cell in each row is the best classifier for that row. The yellow-highlighted cell is the best score in the whole experiment.

K	FS Method	DT	KNN	NB	RF
500	ANOVA-F	0.7827	0.7796	0.7445	0.8376
500	AllFeatures	0.7853	0.7956	0.7479	0.8541
500	Chi2	0.7861	0.7834	0.7467	0.8412
500	InfoGain	0.7540	0.7696	0.7210	0.8320
500	ReliefF	0.7793	0.7657	0.7393	0.8385
2000	ANOVA-F	0.7877	0.8051	0.7741	0.8553
2000	AllFeatures	0.7838	0.8082	0.7775	0.8628
2000	Chi2	0.7869	0.8067	0.7735	0.8566
2000	InfoGain	0.7848	0.8008	0.7482	0.8595
2000	ReliefF	0.7802	0.7738	0.7787	0.8420
5000	ANOVA-F	0.7869	0.8167	0.7897	0.8527
5000	AllFeatures	0.7867	0.8178	0.7973	0.8566
5000	Chi2	0.7853	0.8182	0.7913	0.8545
5000	InfoGain	0.7860	0.8058	0.7786	0.8635
5000	ReliefF	0.7834	0.7905	0.8052	0.8420
10000	ANOVA-F	0.7906	0.8273	0.8048	0.8579
10000	AllFeatures	0.7866	0.8250	0.8105	0.8571
10000	Chi2	0.7900	0.8231	0.8035	0.8534
10000	InfoGain	0.7845	0.8149	0.7956	0.8581

Table 2. F1-score pivot table. ReliefF is missing for K=10,000 by design (see Section 3.4).

5.3 Mean Performance by Classifier

Averaging across all configurations, Random Forest is clearly the strongest model on every metric.

Classifier	Accuracy	Precision	Recall	F1	AUC
DecisionTree	0.7847	0.7904	0.7848	0.7837	0.8095
KNN	0.8028	0.8117	0.8029	0.8015	0.8842
NaiveBayes	0.7758	0.7787	0.7758	0.7752	0.8770
RandomForest	0.8522	0.8601	0.8522	0.8513	0.9398

Table 3. Mean metrics per classifier (averaged over all FS methods and feature sizes).

6. Visualization

6.1 ROC Curves

ROC curves let me compare classifiers at all decision thresholds. The figures below show ROC curves for K=500, 2,000, 5,000, and 10,000 (all using the AllFeatures baseline so the comparison is fair).

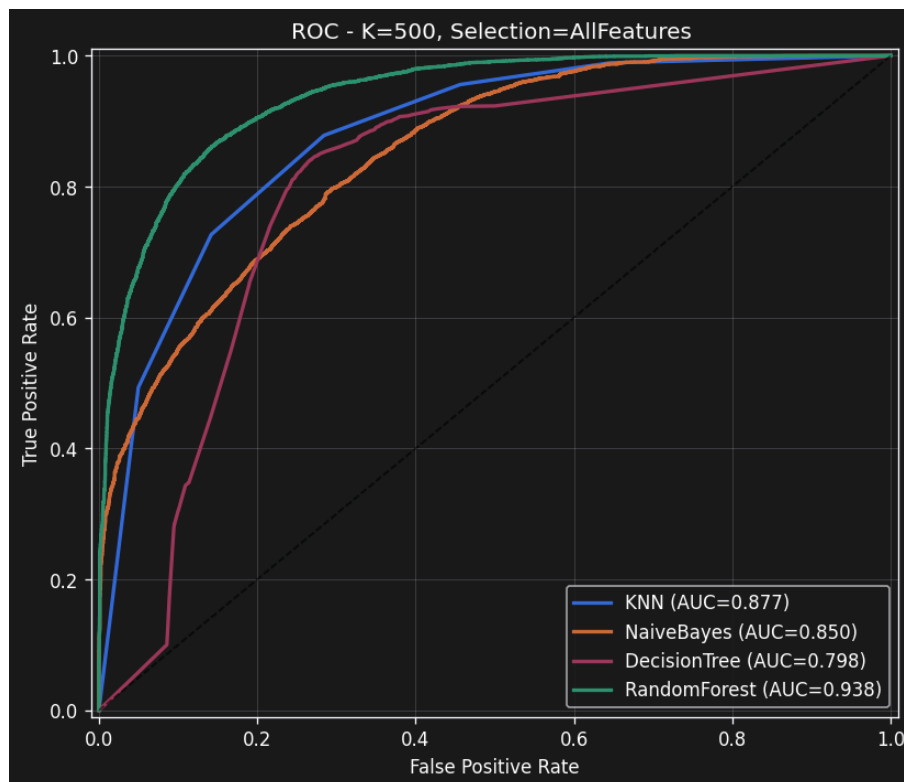


Figure 2. ROC curves at K=500 (baseline).

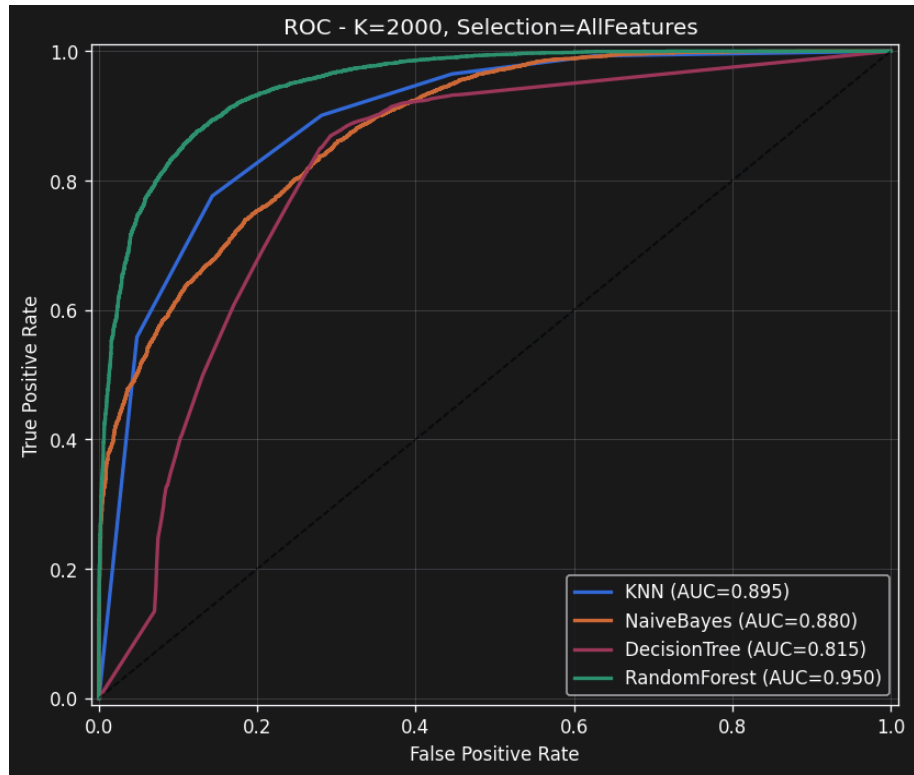


Figure 3. ROC curves at K=2,000 (baseline).

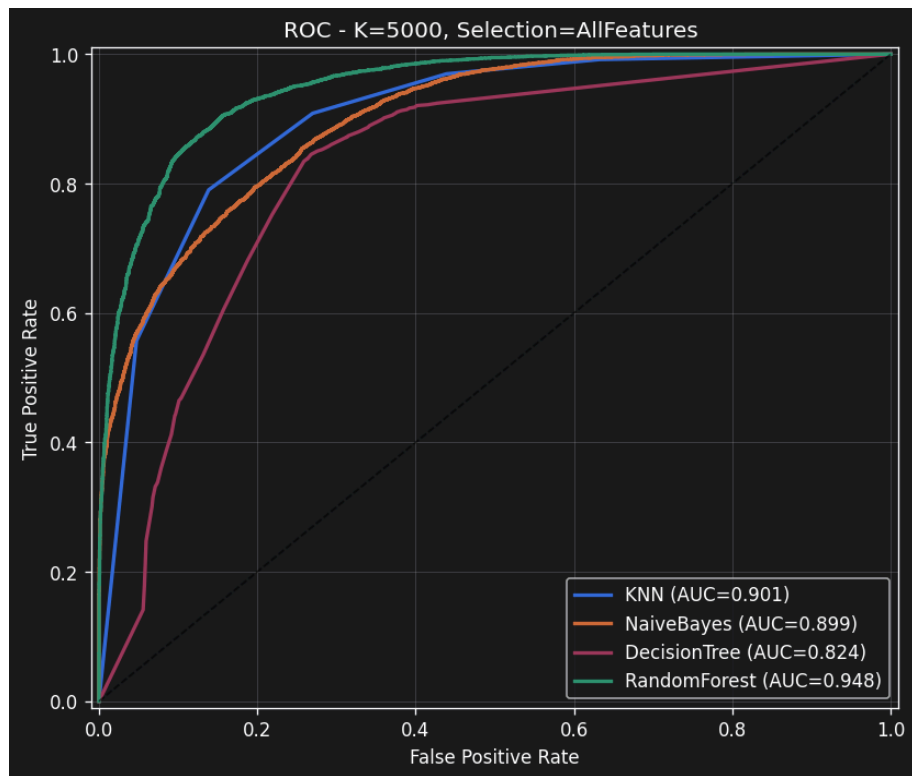


Figure 4. ROC curves at K=5,000 (baseline).

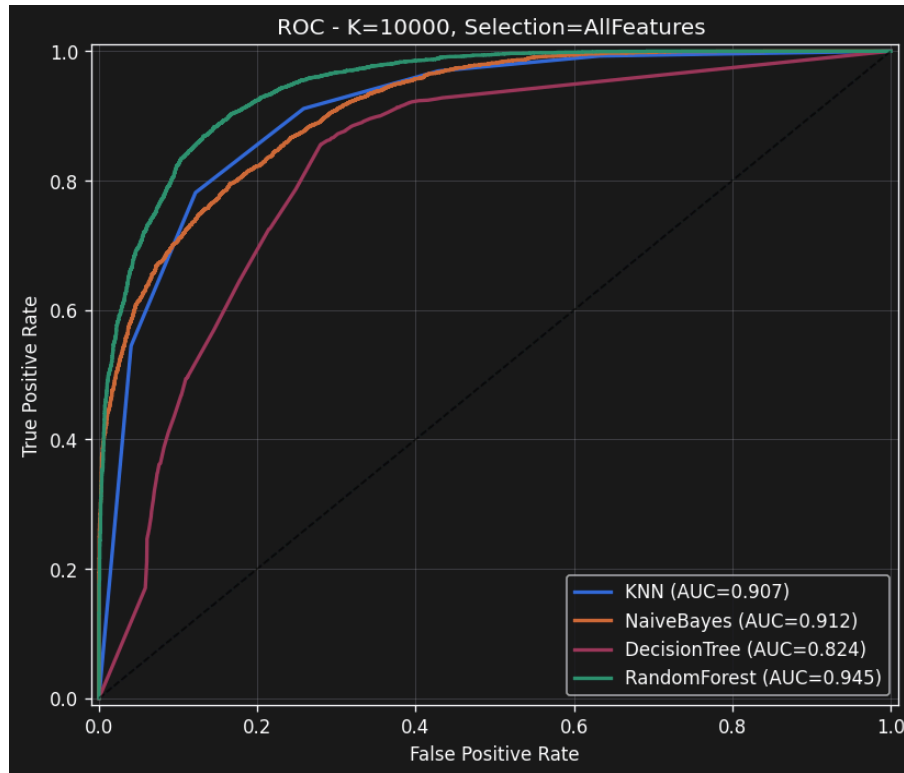


Figure 5. ROC curves at K=10,000 (baseline).

The next figure shows the ROC curves for the best model configuration (K=5,000 with Information Gain)

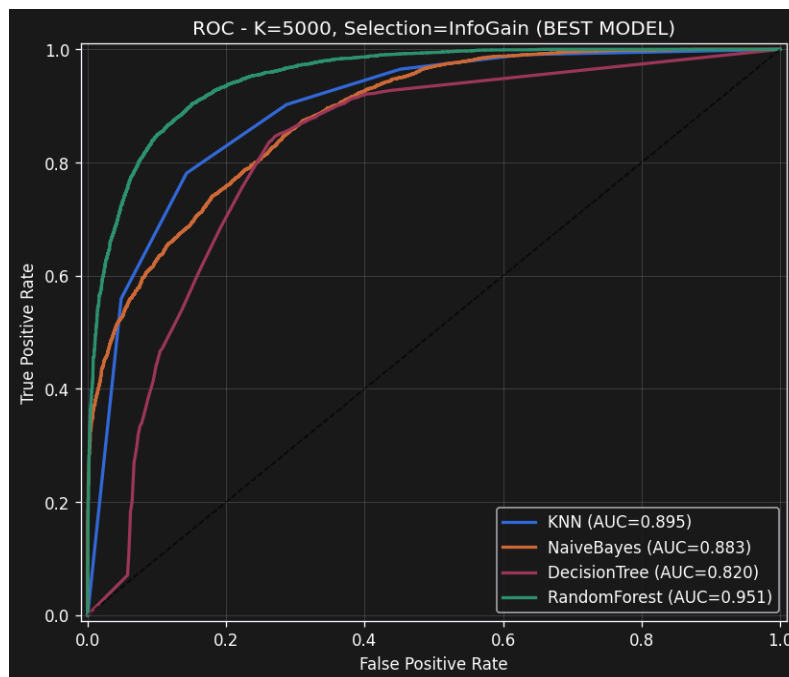


Figure 6. ROC curves for the best configuration: K=5,000 + Information Gain. Random Forest reaches AUC=0.951.

6.2 Word Clouds

Word clouds give a quick visual feeling of which words are common in each class.

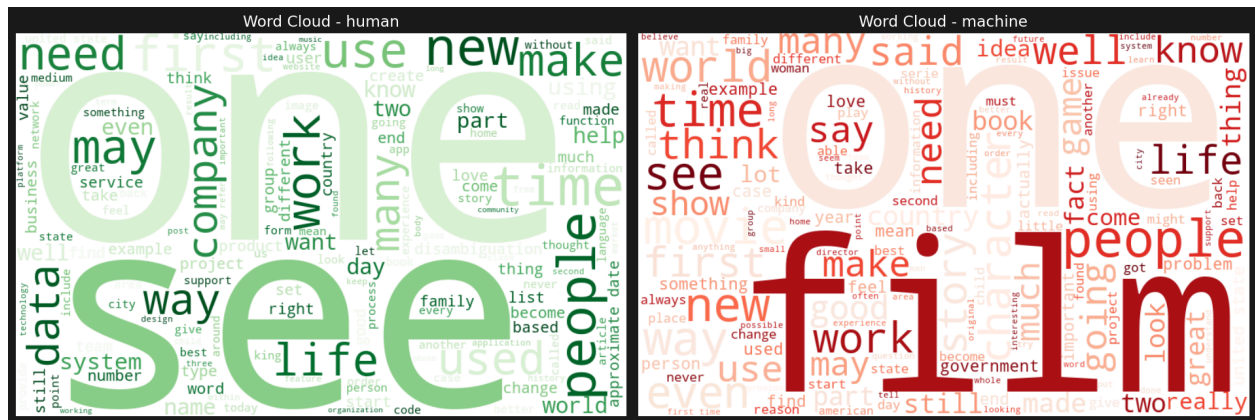


Figure 7. Word clouds for human (left) and machine (right) texts.

6.3 Top 20 Words (Bar Charts)

Word clouds are nice but not exact. The bar charts show the precise frequencies of the most common words in each class.

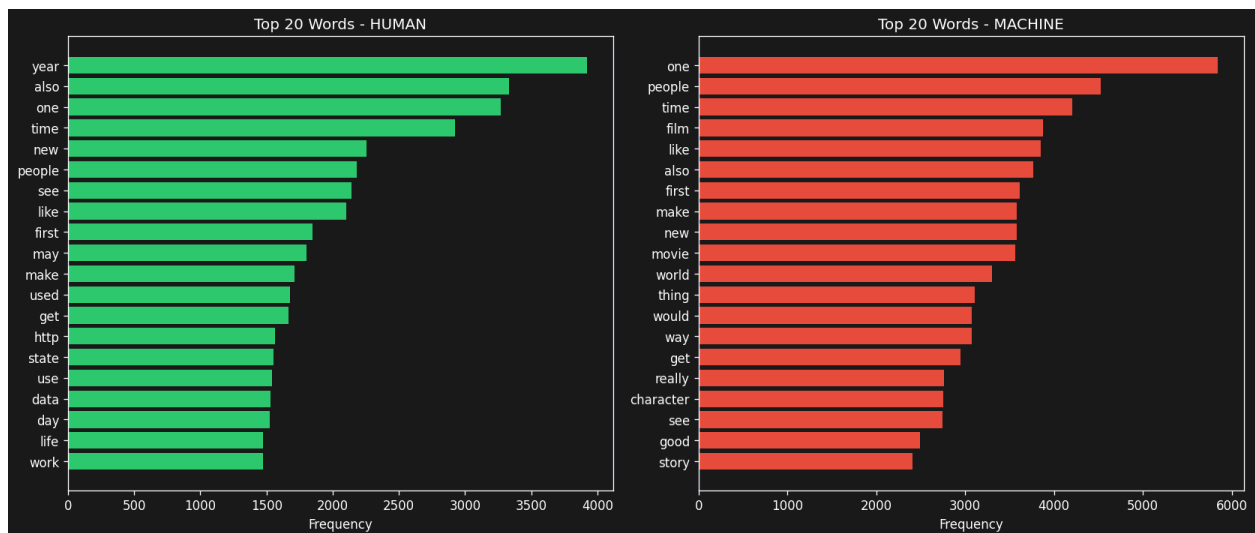


Figure 8. Top 20 most frequent words in human (left) and machine (right) texts.

6.4 Confusion Matrix (Best Model)

Below is the confusion matrix from 5-fold cross validation predictions for the best configuration: Random Forest + K=5,000 + Information Gain.

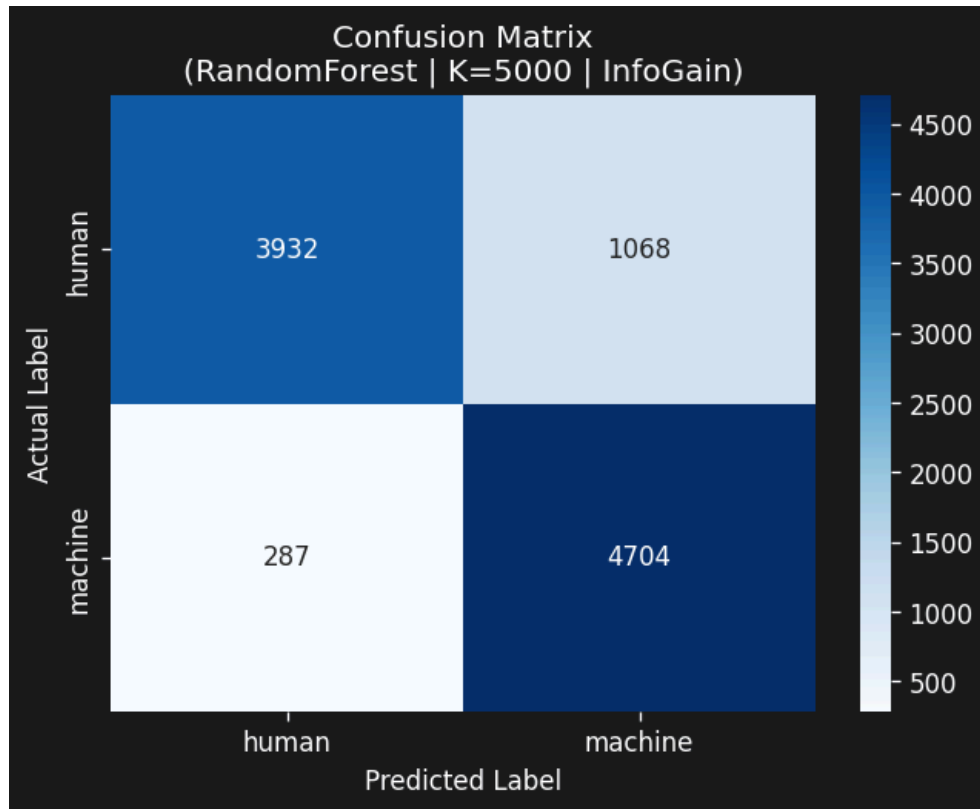


Figure 9. Confusion matrix for the best model (Random Forest, K=5,000, InfoGain).

As a table, the same numbers look like this:

	Predicted: human	Predicted: machine
Actual: human	3932 (TP)	1068 (FN)
Actual: machine	287 (FP)	4704 (TN)

Table 4. Confusion matrix in table form. TP/TN are correct predictions, FP/FN are errors.

From these numbers I can compute the per-class recall:

- **Machine recall:** $4704 / 4991 = 94.3\%$
- **Human recall:** $3932 / 5000 = 78.6\%$

So the model is much better at finding machine texts than at finding human texts.

6.5 Effect of Feature Size on F1

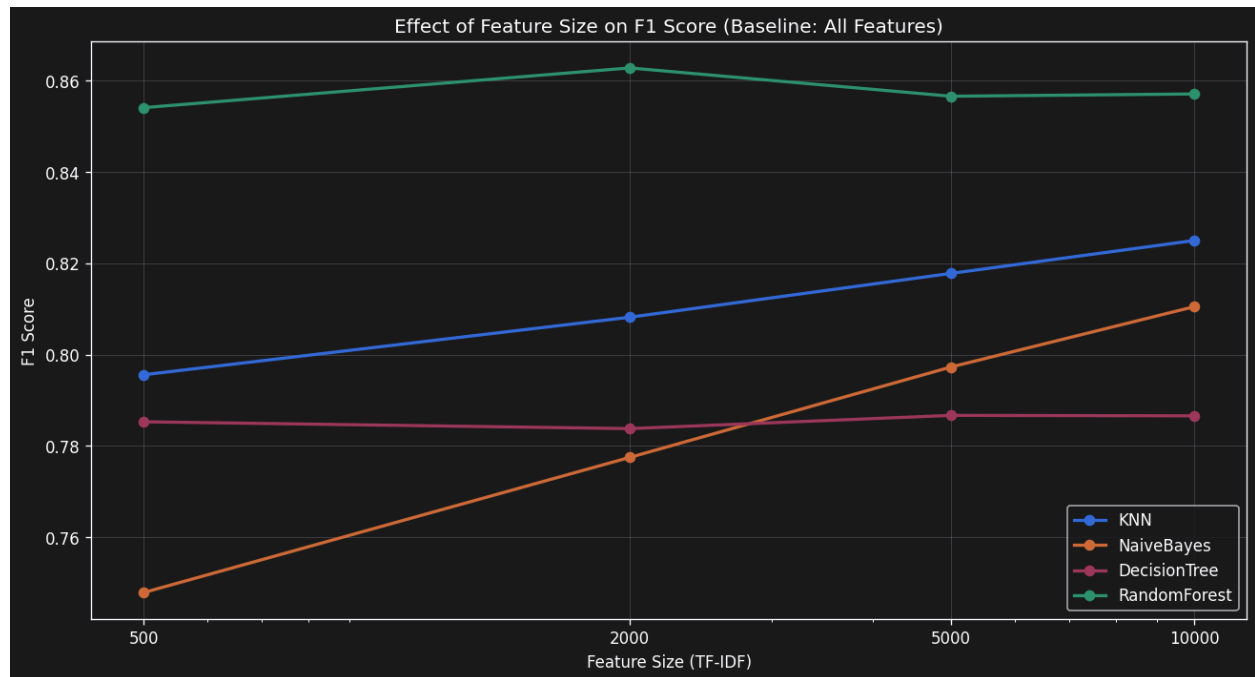


Figure 10. F1 score as a function of feature size for each classifier (FS = AllFeatures baseline).

7. Discussion

7.1 Effect of Feature Size

The four classifiers reacted very differently when I increased the number of features:

- **Naive Bayes:** Benefited the most. F1 went from 0.748 (K=500) up to 0.811 (K=10,000), a gain of about 6 points. MultinomialNB extracts probability information from every single feature, so more features means more information.
- **KNN:** Also improved with more features (0.796 → 0.825). I expected to see a strong "curse of dimensionality" effect but did not. Using cosine distance instead of Euclidean is the likely reason — cosine handles sparse high-dimensional vectors much better.
- **Decision Tree:** Almost flat (0.785 → 0.787). A single tree picks its own splits, so giving it more features did not really help.
- **Random Forest:** Interesting pattern. Big jump from K=500 to K=2,000 (0.854 → 0.863), then a slight drop at K=5,000 and K=10,000 (~0.857). For this dataset, K=2,000 is the "sweet spot" for RF; extra features after that mostly add noise.

7.2 Comparison of Classifiers

- **Random Forest is clearly the best.** All 19 RF configurations had $F1 \geq 0.832$ and $AUC \geq 0.918$. The top 10 list is 100% Random Forest. Best score: $F1=0.8635$, $AUC=0.9509$.
- **KNN comes second.** Best score $F1=0.828$ at $K=10,000$. It catches up with Random Forest only when many features are available.
- **Naive Bayes is third.** Weak at $K=500$ ($F1=0.748$) but reaches $F1=0.811$ at $K=10,000$.
- **Decision Tree is the weakest.** $F1$ stays around 0.78 – 0.79 no matter what. The fact that Random Forest is about 7 $F1$ points above a single tree clearly shows the benefit of the ensemble approach.

7.3 Effect of Feature Selection

The feature selection results were partly surprising:

- **AllFeatures (baseline) is competitive.** Removing the explicit FS step often gave equal or better scores than applying ReliefF / χ^2 / InfoGain / ANOVA. This is because TF-IDF's `max_features` parameter already does a kind of implicit feature selection.
- **Information Gain is the best for Random Forest.** It produced the absolute best result ($K=5,000$ + RF + InfoGain $\rightarrow F1=0.8635$). However, it was the worst FS method for Naive Bayes ($F1=0.721$ at $K=500$ vs χ^2 0.747).
- **Chi-Square and ANOVA F-test are very close.** They are fast, stable, and never cause big drops. Good safe defaults.
- **ReliefF underperformed.** On average it was 1–2 $F1$ points below the filter methods, and 3–4 times slower (~ 55 – 60 s per fold vs ~ 17 s). One exception: ReliefF + Naive Bayes at $K=5,000$ reached $F1=0.805$. This matches the literature: ReliefF shines on structured low-to-medium dimensional data with feature interactions, not on high-dimensional sparse text.

7.4 Confusion Matrix Analysis

Looking at the best model's confusion matrix (Section 6.4), the error profile is asymmetric:

- **Machine recall = 94.3%** — very good, the model rarely misses an AI-generated text.
- **Human recall = 78.6%** — much worse. About 21% of real human texts are misclassified as machine.

Practically, this is risky for an AI-detection tool: too many real human writers would be falsely flagged as AI users. In a real product I would adjust the decision threshold (using `predict_proba`) to balance the two error types based on the use case.

7.5 Interpretation of Important Features

The top-20 word charts and the word clouds show interesting patterns about what makes the two classes different:

- **Human texts** use words like year, data, http, state, work, life, company, system. These are real-world, technical, and often informational terms (likely from Wikipedia or technical articles).
- **Machine texts** are dominated by film, movie, character, story, world, like, really. These are narrative/entertainment words, suggesting the AI texts were generated mostly from prompts about films, fiction, or reviews.

This thematic gap is the strongest signal the classifier learned. Together with the length difference (machine peak around 400–450 words), it explains why even simple TF-IDF + classical ML can reach $F1 \approx 0.86$. The downside is that this model probably learned topic and length differences, not deep linguistic style differences. On a balanced topic distribution, the same model might perform worse — this is a real-world generalization risk.

7.6 General Conclusion

- **Best configuration:** Random Forest + K=5,000 + Information Gain. $F1=0.8635$, $AUC=0.9509$, $Accuracy=0.8644$.
- **Robustness:** Random Forest beat all other classifiers under every feature size and every FS method. It is the safe choice for this kind of text classification.
- **Feature selection:** Filter methods (χ^2 , InfoGain, ANOVA) are practically free in compute. ReliefF is too expensive on high-dimensional sparse text and did not improve results enough to justify its cost.
- **Future work:** Try transformer-based embeddings (BERT, RoBERTa) or stylometric features (sentence length variance, perplexity, burstiness) for a stronger and more topic-independent detector.

8. Implementation Notes

- **Language / environment:** Python 3, Jupyter Notebook.
- **Main libraries:** scikit-learn, NLTK, skrebate (ReliefF), pandas, numpy, matplotlib, seaborn, wordcloud, openpyxl.
- **Reproducibility:** All random seeds fixed to 42. StratifiedKFold uses shuffle=True with the same seed.
- **Data leakage prevention:** TF-IDF and feature selection are fit only on the training fold inside each CV iteration.
- **Checkpointing:** After every model is finished, the running results table is written to results_checkpoint.xlsx so a crash does not lose all data.
- **Memory:** gc.collect() is called after each model. ReliefF uses subsampling (1,500 samples) and stays in sparse format for transform.